

A PRACTICAL “RULES OF THUMB”

- **Rule 1: Every module gets its own testbench early**
Don't wait until you've connected five modules and then debug the mess.
- **Rule 2: Dump waveforms by default**
GTKWave exists for a reason; use it.
- **Rule 3: Sequential logic uses non-blocking assignments**
Keep your clocked always blocks clean and predictable.
- **Rule 4: Combinational always blocks get defaults**
Avoid unintended latches.
- **Rule 5: Verify “behaviour,” not just “it runs”**
A simulation that runs without errors can still be wrong.

```
initial begin
    $dumpfile("waves/uart_tx.vcd");
    $dumpvars(0, tb_uart_tx);

    rst_n = 0; send = 0; data = 8'h00;
    #5000;
    rst_n = 1;

    // Send 0x55 (01010101)
    data = 8'h55;
    #2000;
    send = 1;
    #1000;
    send = 0;

    #200000;
    $finish;
end
endmodule
```

Step-by-step: decode UART in the waveform

1. Open `waves/uart_tx.vcd` in GTKWave
 2. Find `tx` and zoom in
 3. Identify:
 - idle high line
 - start bit low
 - 8 data bits LSB-first
 - stop bit high
- Once you can “read” UART visually, you’ll feel unstoppable on real hardware.

Verification-lite: start acting like a grown-up

You don't need industrial-grade verification methodology to be effective today. But you *do* need checks.

Add simple self-checks in testbenches

For adder:

```
if ({cout, sum} != (a + b + cin))
    $display("Mismatch!");
```

For counter:

- check it resets to 0
- check it increments when enabled
- check it holds when disabled

The “X discipline”

If your sim shows X, treat it like a bug, not a shrug.

- initialize regs in testbenches
- reset designs properly
- ensure all combinational outputs are fully assigned

Tool upgrade path: when Verilator becomes useful

Icarus is great for learning and general Verilog simulation. Verilator is different: it converts Verilog/SystemVerilog into a C++/SystemC model and runs it like compiled code, making it very fast for certain workloads.

Verilator's official installation guide provides multiple install paths and highlights that distro packages may be older.

Use Verilator when:

- your simulation is slow
- you want stricter compile-time checking
- you're building more complex test harnesses

But for this Fast Track flow, Icarus + GTKWave is the quickest confidence builder.

Quick reference: the commands you'll keep typing

```
# Compile
iverilog -o build/tb_name tb/tb_name.v src/
design.v

# Run
vvp build/tb_name

# View waveforms
gtkwave waves/name.vcd
```

And if you want to explore Verilator later:

- follow its install guide (package manager quick install or git quick install)
- remember it's a compiler-style flow, not a traditional interpreter-style simulator 